# Data science and data exploration

Dr. Leonid Serkin
National Autonomous University of Mexico (UNAM)

## Examples from the lecture

You are invited to run each of the examples shown during the lecture. Explore them at your own pace. Just follow the "README" at:

`https://github.com/lserkin/csc2026-lecture`

After 1) accessing SWAN, 2) configuring your SWAN session and 3) downloading the project repository, you can run the example notebooks. Once the download is complete, you will see yourself within a directory named "csc2026-lecture".

Pick the notebook (.ipynb file) you want and launch it (click), a new window will be opened. Run the notebook by clicking the "Run" button repeatedly until all cells have finished executing. You can also run everything by going into "CEL" and then"Run Al".

Instructions are embedded in each file. For some exercises, there are also data files that you can browse by opening them in SWAN.

The recommended order of the notebooks is the following

1. *Preprocessing.ipynb* – Start simple with data pre-processing.
2. *Datasaurus.ipynb* – A proof that summary statistics are not enough!
3. *Wine.ipynb* – All about Exploratory Data Analysis (EDA).
4. *Color.ipynb* – A study about color maps.
5. *Dimensionality.ipynb* – Examples about dimensionality reduction.
6. *GLT_10cities.ipynb* – Missing values in the Global Land Temperature dataset.
7. *SMOTE.ipynb* – An example about Synthetic Minority Oversampling Technique (SMOTE).
8. *Bootstrap.ipynb* – Resampling method with replacement.
9. *Cluster.ipynb* – An example of cluster visualizations.
10. *Wordcloud.ipynb* – Building word clouds and co-occurrences.

# Additional exercises

The next exercises are designed to help you get started with data science and interactive data exploration. Explore them at your own pace. Just follow the "README" at:

`https://github.com/lserkin/csc2026-exercise`

After 1) accessing SWAN, 2) configuring your SWAN session and 3) downloading the project repository, you will see yourself within a directory named "csc2026-exercise". There you will find three initial notebooks – one for each exercise. Each serves as a placeholder for your code.

## Exercise 1: Higgs dataset exploration

For this exercise, you will apply data exploration techniques to the *Higgs* dataset introduced in the paper *"Searching for Exotic Particles in High-Energy Physics with Deep Learning"* by P. Baldi, P. Sadowski, and D. Whiteson (arXiv:1402.4735 [hep-ph]. This paper is one of the first demonstrations in which Deep Learning methods were shown to outperform traditional Machine Learning techniques in a realistic HEP task. The dataset we will use is a binary classification problem and the goal is to distinguish between a *signal* process involving Higgs bosons:

$$gg \to H^0 \to W^{\mp}H^{\pm} \to W^{\mp}W^{\pm}h \to W^{\mp}W^{\pm}b\bar{b}.$$

from the *background* process that produces the same final-state as the signal process, but without an intermediate Higgs boson:

$$t\bar{t} \to W^{\mp}W^{\pm}b\bar{b},$$

However, when obtaining the dataset from the authors, it was "mysteriously" corrupted in transit. As a result, you will need to explore the kinematic features using data visualization techniques and use your intuition to understand what went wrong.

1. Start by loading the dataset and perform a first sanity check:

```
import pandas as pd
df = pd.read_csv("Higgs.csv")
print(df.shape)
print(df.head())
print(df.dtypes) # Check the data types stored!
```

2. Make a simple histogram:

```
import matplotlib.pyplot as plt
plt.hist(df[df.columns[1]], bins=50)
plt.show()
```

Can you spot if something is wrong with it? Do you agree that we need additional validation steps?

3. Now let's force a strict numeric operation:

```
col = df.columns[1]
pd.to_numeric(df[col])
```

Can you explain why it fails?

4. Well, we do see that data is corrupt! For each column, count how many entries cannot be converted to numeric[1].

```
bad_counts = {}
for col in df.columns:
    x = pd.to_numeric(df[col], errors="coerce")
    bad_counts[col] = x.isna().sum()

print(pd.Series(bad_counts).sort_values(ascending=False))
```

5. Print a few corrupt values from one column:

```
col = df.columns[1]
x = pd.to_numeric(df[col], errors="coerce")
print(df.loc[x.isna(), col].head(10))
```

6. And now clean the dataset by removing trailing symbols and converting to numeric:

```
df_clean = df.copy()
for col in df_clean.columns:
    df_clean[col] = (df_clean[col].astype(str)
                     .str.replace(r"[#!%&*]$", "", regex=True))
    df_clean[col] = pd.to_numeric(df_clean[col], errors="raise")
```

7. Verify that cleaning succeeded:

```
print(df_clean.dtypes)
print("Total NaNs:", df_clean.isna().sum().sum())
```

If you obtained

```
Total NaNs: 0
```

it worked!

[1] When copy-pasting from this PDF, watch out for indentation in Python!

8. Explain why a histogram can appear to work even when the column contains strings.

9. Now visualize the kinematics of all the variables. Draw the histograms normalized to the *fraction of events* so that each distribution integrates to unity, allowing a direct comparison of shapes independent of class imbalance:

   ```
   w_bkg = np.ones(len(df_background)) / len(df_background)
   w_sig = np.ones(len(df_signal)) / len(df_signal)
   ```

   where

   ```
   df_signal     = df_clean[df_clean["label"] == 1]
   df_background = df_clean[df_clean["label"] == 0]
   ```

   Iterate simultaneously over subplots and feature names, and make it nice by drawing the distributions using unfilled step histograms, with background shown as a black solid line and signal as a red dashed line:

   ```
   for ax, col in zip(axes, features):
       ax.hist(df_background[col], bins=50, weights=w_bkg,
               histtype="step", color="black")
       ax.hist(df_signal[col], bins=50, weights=w_sig,
               histtype="step", color="red", linestyle="--")
   ```

10. We are done! But, are we? At this stage, carefully inspect the distributions of all kinematic variables. Ask yourself whether every feature behaves as you would physically expect, and pay particular attention to quantities that are subject to basic physical constraints (corrupt laws of physics, OMG!)

11. Now that you found it, ask yourself: which one is the most discriminating variable? You can do it by eye, or use a simple and widely used option: the Kolmogorov–Smirnov (KS) statistic. For each kinematic variable, apply the two-sample KS test to the signal and background distributions and record the value of the KS statistic:

    ```
    from scipy.stats import ks_2samp
    ks_scores = {}
    for col in features:
        ks_scores[col] = ks_2samp(df_background[col],
                                  df_signal[col]).statistic
    ranking_ks = pd.Series(ks_scores).sort_values(ascending=False)
    print(ranking_ks.head(5))
    ```

    The variables are then ranked according to their KS statistic, with larger values indicating stronger separation between signal and background.

## Exercise 2: Global Land Temperature

For this exercise, you will use a much larger Global Land Temperature dataset:

```
import pandas as pd
df = pd.read_csv('GLT_full.csv', encoding='utf-8')
```

Take as baseline the notebook "GLT_10cities.ipynb" from the lecture and apply the missing values treatment we did, i.e. `def fill_missing(values)` to this dataset and proceed to study it:

1. There is an anomaly in the data distribution. With the help of Matplotlib, plot the distribution of the land average temperatures for Rome and Bangkok. As you can see, Rome and Bangkok have very different temperature distributions. However, what is strange is the large difference in the magnitude of their temperatures. What do you think might have happened here? Use your creativity to analyze it and propose an explanation.

2. Now that you have found it, compute the annual average temperature for the any 5 cities you prefer (there are 100 in total). Do you observe any increase over the entire measurement period? Support your answer with plots. And what if you now split the data into periods of 5 or 10 years and observe how the average temperatures have changed in those periods?

## Exercise 3: unknown realistic dataset

For this exercise, you will apply the pre-processing steps a data scientist would do to a large dataset of unknown origin:

```
import pandas as pd
data = pd.read_csv("dataset_full.csv")
```

Start by exploring the general structure of the dataset and carefully check for any extreme values, missing data, or duplicate rows you should take into account. Pay attention to columns that contain strange symbols or unusual text.

Identify data quality issues that could affect your future machine learning model's learning. Explain how you detected them and why they matter. Propose concrete solutions: cleaning, transforming, augmenting, or even removing variables, always with a clear justification.